Karger's Min Cut

Here, we discuss a randomised polynomial time algorithm to compute a cut of minimum size in an unweighted undirected graph.

Randomized Algorithms

A randomized algorithm is an algorithm which uses randomness to make some choice in its procedure. This means, the algorithm uses some source of random bits to guide its behaviour, in hopes of achieving **good performance** (in terms of **running time** or **correctness**), in the *average case* over all possible random choices made.

Thus, for a randomized algorithm, either the running time of the algorithm, or the output of the algorithm, (or both) will be a random variable.

We can thus broadly classify randomized algorithms into two categories:

1. ***Las-Vegas* type randomized algorithms**: These randomized algorithms always output a correct result (so their output is deterministic), but their running time depends on the random bits used. (The time complexity is a random variable.)
2. ***Monte-Carlo* type randomized algorithms**: These randomized algorithms run in similar time regardless of the random bits (deterministic time complexity), however it may sometimes answer wrongly. (The output is a random variable.)

An example of a *Las-Vegas* type randomized algorithm is Quicksort. For these types of algorithms, we study their expected time complexity.

The algorithms that we will discuss here are examples of *Monte-Carlo* type randomized algorithms. For these types of algorithms, we study the probability that the algorithm outputs a correct (or a wrong) result.

Setup

Let $G = (V, E)$ be an undirected unweighted graph, G may have parallel edges (but no self-loops).

A **cut** of G is a non-empty strict subset $U \subsetneq V, U \neq \phi$ of the vertices. The **size of a cut** $w(U)$ is the number of edges whose exactly one end point belongs to U, i.e.

$$w(U) = |\{\{x, y\} \in E \mid x \in U, y \notin U\}|$$

The **min-cut** problem is computing a cut of minimum size in G. Alternatively, the size of a min-cut is the minimum number of edges to be removed from G to make it disjoint.

We can give a flow-based algorithm that finds the min-cut which runs in polynomial time. Here we discuss a randomised algorithm for solving this problem.

Karger's Randomized Min-Cut Algorithm

Edge Contraction

For a graph $G = (V, E)$, let $e = \{x, y\} \in E$ be an edge. Then, define the operation **edge-contraction** of e in G as follows:

1. Remove the vertices x and y (and all their edges).

2. Add a new vertex v_{xy}. Add edges from v_{xy} to all vertices which were earlier adjacent to x or y. (this may introduce parallel edges). Don't add any self-loop.

Call the resulting graph G/e.

The algorithm

KARGER-MIN-CUT(G)

```
1  while |V| > 2
2      pick an edge e uniformly at random
3      contract e in G
4  let U be the set of vertices that contracted to form one of the remaining vertices
5  return U
```

On the graph G, continuously contract edges randomly. Do so until you are left with 2 vertices. Return this cut as the min cut of G.

(Yes, you read that right, that's exactly what the algorithm is!)

Error Bound

Ok, so the algorithm won't always find the min-cut, it might return a cut which isn't the min-cut. We want to get a bound on the probability that the cut found is not a min-cut, i.e. the algorithm reports an incorrect output.

Some Observations

Consider a cut S in G, and an edge $\{u, v\} \in E$. Say, that the cut S is **killed** by e if u and v are on the opposite sides of the cut, i.e. $u \in S, v \notin S$ or $u \notin S, v \in S$. Otherwise (if both vertices are on the same side of the cut), say that cut S **survives** e.

This notion indicates, that if we ever contract the edge $e = \{u, v\}$, then whatever the final cut will be obtained, necessarily u and v must be on the same side of the cut (since during contraction, we merged them into the same vertex). So any cut S where u and v are on the opposite sides are *killed*, in the sense that they are eliminated from the list of potential cuts that the algorithm could have output.

Then, we have the following observation:

> **Contraction preserves surviving cuts**
>
> Let $G = (V, E)$ be an undirected graph. Let $G' = G/e$ for some $e \in E$. The cuts of G' are in one-to-one correspondence with the cuts of G that survived e, with corresponding cuts having the same size.

Proof: Let S be a cut that survived $e = \{x, y\}$, so assume WLOG, $x, y \in S$, then $S' = (S \setminus \{x, y\}) \cup \{v_{xy}\}$ is the corresponding cut in G'. Conversely, if S' is a cut of G' with $v_{xy} \in S'$, then $S = \left(S' \setminus \{v_{xy}\}\right) \cup \{x, y\}$ is the corresponding surviving cut of G. By definition of edge contractions, all edges are preserved (other than the $\{x, y\}$ edge), so their sizes are the same. □

Thus, by applying this lemma repeatedly, we get the following observation: **after a sequence of contractions, the cuts of the current graph are exactly the cuts of G that survived every edge contraction.**

Another observation that we will use is the **handshaking lemma**: In a graph G with n vertices, suppose the size of the min-cut is k. Then, for any vertex v, its degree must be at least k (why?). Then, the total number of edges in the graph is at least $nk/2$.

Calculation

Let k be the size of a min-cut of G. Fix a min-cut X, and let F be the set of the k edges crossing X. Let G_i denote the graph when i vertices remain, so $G_n = G$ and the algorithm stops at G_2.

Let e_i be the edge that was contracted at stage G_i. (So the first edge contracted is e_n and the last edge contracted is e_3.)

Observation. The run outputs X if and only if none of $e_n, e_{n-1}, \ldots, e_3$ are in F.

- If some contracted edge lies in F, it kills X, so X cannot be the output.
- If none does, X survives all the contractions. By the lemma, X is a cut of G_2. But G_2 has two vertices, so it has exactly one cut, and that cut must be X.

Thus, the probability of the algorithm outputting a fixed min-cut X is exactly the probability that no $e_i \in F$.

Initially, the graph has $m = |E|$ edges. The probability that the first edge chosen for contraction is not in F is

$$p_n = 1 - \frac{k}{m}$$

Now, we know that $m \geq \frac{nk}{2}$. So,

$$p_n \geq 1 - \frac{k}{\frac{nk}{2}} = 1 - \frac{2}{n} = \frac{n-2}{n}$$

Similarly, the probability that the second edge chosen is not in F is $p_{n-1} = \frac{n-3}{n-1}$, and so on.

So the overall probability that no contracted edge kills X is

$$\prod_{i=3}^{n} p_i \geq \frac{n-2}{n} \times \frac{n-3}{n-1} \times \ldots \times \frac{2}{4} \times \frac{1}{3} = \frac{2}{n(n-1)}.$$

A single run of Karger's algorithm returns any fixed min-cut X with probability at least $2/(n(n-1))$. Hence it returns *some* min-cut with at least this probability, and thus the error probability is

$$p_{\text{fail}} \leq 1 - \frac{2}{n(n-1)}.$$

Improving the error bound

For large n this is a poor bound: the failure probability approaches 1 asymptotically. Can we do better?

Consider the following procedure:

1. Run KARGER-MIN-CUT on G independently c times, for some value of c.
2. Return the smallest cut found among the c runs.

The procedure fails only if *none* of the c runs finds a min-cut. The runs use independent random bits, so this probability is the product of the individual failure probabilities:

$$p_{\text{fail}}^c \leq \left(1 - \frac{2}{n(n-1)}\right)^c.$$

How large should c be to make this failure probability our desired (constant) value?

If we choose $c = (n(n-1))/2$, we get

$$p_{\text{fail}}^c \leq \left(1 - \frac{2}{n(n-1)}\right)^{n(n-1)/2} = \left(1 - \frac{1}{c}\right)^c \leq \frac{1}{e},$$

using the inequality $1 - x \leq e^{-x}$ for all real x. So with $c = n(n-1)/2 = O(n^2)$ repetitions[1], the algorithm finds a min-cut with probability at least $1 - 1/e \approx 0.63$. (We can get to any arbitrary constant probability by multiplying c with a constant value.)

Time Complexity

By representing the graph G as an adjacency matrix, one edge contraction step can be performed in $O(n)$ time.

Then, one run of KARGER-MIN-CUT requires $n - 2$ edge contractions, so it takes $O(n^2)$ time.

For achieving a constant error bound, we need to repeat the procedure for $O(n^2)$ times. So the overall time complexity of this algorithm is $O(n^4)$.

An unrelated result

This leads to an interesting corollary, on the number of min-cuts in a graph. Suppose a graph G has k min-cuts, X_1, X_2, ..., X_k. For each X_i, the probability that the algorithm outputs X_i is $\frac{2}{n(n-1)}$. Since the algorithm will output exactly one min-cut, these probabilities are disjoint, and can be added:

$$\sum_{i=1}^{k} P(X_i) = \frac{2k}{n(n-1)} \leq 1$$

$$\Rightarrow k \leq \frac{n(n-1)}{2} = \binom{n}{2}$$

So, $\binom{n}{2}$ is an upper bound on the number of min-cuts that a graph can have. (This bound is actually tight, and is realised in cycle graphs.)

[1] Observe, that we bounded the success probability of one run to $\Omega\left(\frac{1}{n^2}\right)$, so we require $O(n^2)$ repetitions to get a constant success probability. This is a general idea in randomised algorithm design: We try to get the success probability to the order of reciprocal of some polynomial in n, then we can achieve a constant success probability within polynomial number of repititions.

Karger-Stein Algorithm for Min-Cut

An extension of the above algorithm due to Karger and Stein achieves an order of magnitude improvement.

Key Observation

The analysis of Karger's algorithm shows that the early contractions are very safe, while the later ones are risky. When the graph has i vertices, the probability that the contracted edge does not lie in F (i.e. min-cut X is not killed) is at least $\frac{i-2}{i}$. This is close to 1 for large i, but only $\frac{1}{3}$ when $i = 3$.

Let's only perform the edge contractions for the first few contractions, say till t vertices remain. Thus the probability that X survives the contractions from n vertices down to t vertices is at least

$$\prod_{i=t+1}^{n} \frac{i-2}{i} = \frac{n-2}{n} \times \frac{n-3}{n-1} \times \ldots \times \frac{t-1}{t+1} = \frac{t(t-1)}{n(n-1)}.$$

Now choose $t = 1 + n/\sqrt{2}$, i.e. perform $n - n/\sqrt{2} - 1$ contractions. Then $t(t-1) \geq \left(1 + n/\sqrt{2}\right)\left(n/\sqrt{2}\right) \geq n^2/2$, so

$$P\Big(\text{X survives the first } n - n/\sqrt{2} - 1 \text{ contractions}\Big) \geq \frac{n^2/2}{n(n-1)} \geq \frac{1}{2}.$$

Hence, the probability that Karger's algorithm does *not* output the cut X is small (at most $1/2$) until the first $n - n/\sqrt{2} - 1$ iterations, and grows rapidly after that. The idea of Karger and Stein is to run Karger's algorithm for $n - n/\sqrt{2} - 1$ iterations, and then recurse on the remaining graph twice, and keep the best solution.

The algorithm

KARGER-STEIN(G)

1 **if** $|V| = 2$: **return** the cut
2 run the contraction algorithm on G for $n - n/\sqrt{2} - 1$ iterations; let G' be the resulting graph
3 $U_1 \leftarrow$ KARGER-STEIN(G')
4 $U_2 \leftarrow$ KARGER-STEIN(G')
5 **return** the better of U_1 and U_2

Here, U_1 and U_2 are computed by two independent recursive calls on the same contracted graph G' (each makes its own random choices).

Time Complexity

Let us first analyse the running time $T(n)$ of the algorithm. Using an adjacency matrix, we can perform each contraction in $O(n)$ time. Hence, we have the following recurrence:

$$T(n) = \begin{cases} 2T\left(n/\sqrt{2}\right) + O(n^2) & \text{when } n \geq 3 \\ O(1) & \text{otherwise.} \end{cases}$$

The solution of the above recurrence, using the master's theorem, is

$$T(n) = O(n^2 \log n).$$

Error Bound

As before, let X be a min-cut of G with edges F, we want the probability that the algorithm returns X. For this, we need to analyse the recursion tree for this algorithm.

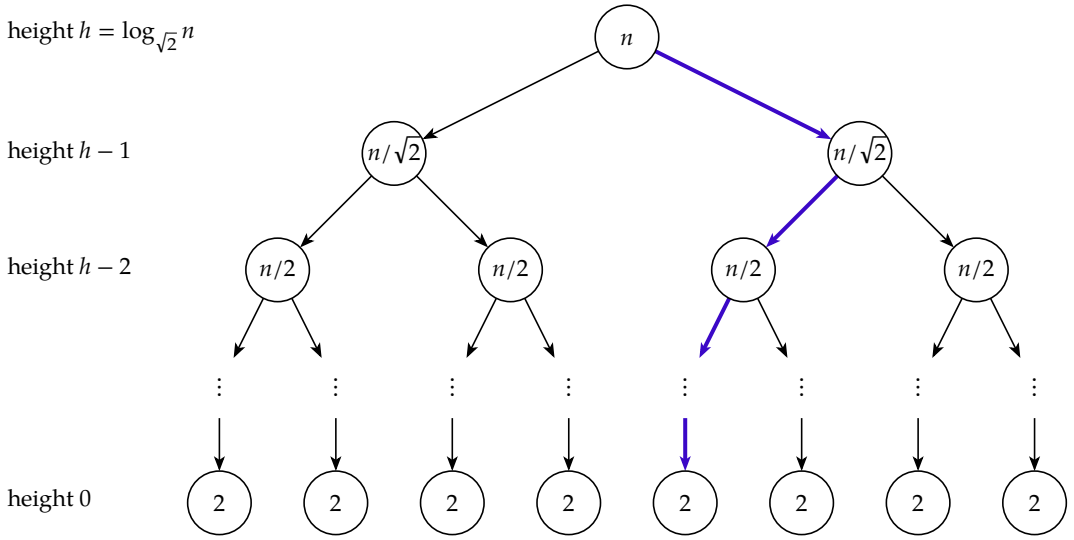


Figure 1: Recursion tree for a run of KARGER-STEIN

Each recursive call to KARGER-STEIN is represented by a node in the recursion tree. Since each call makes two recursive calls (other than the base case), the tree is a full binary tree. The recursive calls are made with a graph with $n/\sqrt{2}$ nodes, if the original graph had n nodes. At the base case, the graph only has 2 nodes. Note that the input graph to both recursive calls made by a node is the same, only the random choices made by the two children differ.

At each non-leaf node, edge contractions are performed (randomly) to get the graph from n nodes to $n/\sqrt{2}$ nodes. Each leaf node returns a cut. Finally, the output of the algorithm is the best cut of all the cuts returned by the leaf nodes.

We want to find the probability that the algorithm returns the cut X. For this to happen, there must be at least one leaf node which returns the cut X, so this means that X survived all the edge contractions that happened in the path from the root node to this leaf.

Observation: At any non-leaf node, the probability that X survives all the edge contractions that happen at this node is at least $\frac{1}{2}$.

(This follows from our original observation for the KS algorithm: we only perform enough contractions so that the probability of X getting killed does not exceed $\frac{1}{2}$).

For some non-leaf node v, let $p(v)$ denote the probability that there exists some path from v to a leaf node, such that X survived every edge contraction on that path (call such a path a **good path**).

Let the left child and right child of v be v_L and v_R. Then we can find $p(v)$ as:

$$\begin{aligned} p(v) &= P(X \text{ survived edge contractions in } v) \times P(\text{either } v_L \text{ or } v_R \text{ has a good path}) \\ &\geq \frac{1}{2} \times (1 - P(\text{neither } v_L \text{ nor } v_R \text{ has a good path})) \\ &= \frac{1}{2} \times (1 - P(v_L \text{ does not have a good path}) \times P(v_R \text{ does not have a good path})) \end{aligned}$$

Now, observe by symmetry, $p(v)$ must be same for all nodes at the same height. So, let's denote by $p(d) = p(v)$, for any node v at a height d. Set $p(0) = 1$.

Then,

$$p(d) \geq \begin{cases} \frac{1}{2} \times \left(1 - (1 - p(d-1))^2\right) & \text{if } d > 0 \\ 1 & \text{if } d = 0 \end{cases}$$

Simplifying the recurrence for $d > 0$,

$$p(d) \geq \frac{1}{2}(1 - (1 - p(d-1))^2) = \frac{1}{2}(2p(d-1) - p(d-1)^2) = p(d-1) - \frac{1}{2}p(d-1)^2.$$

Claim: $p(d) \geq \frac{1}{d+2}$.

Proof: By induction on d. For $d = 0$ we have $p(0) = 1 \geq 1/2$. For $d > 0$, assume $p(d-1) \geq 1/(d+1)$. The function $f(y) = y - y^2/2$ is increasing on $[0,1]$ (since $f'(y) = 1 - y \geq 0$), and $p(d) = f(p(d-1))$, so

$$p(d) \geq f\left(\frac{1}{d+1}\right) = \frac{1}{d+1} - \frac{1}{2(d+1)^2} \geq \frac{1}{d+1} - \frac{1}{(d+1)(d+2)} = \frac{1}{d+2}$$

where the last inequality uses $2(d+1) \geq d+2$. □

The root of the recursion tree has height $h = \log_{\sqrt{2}} n = 2\log_2 n$, and its input is G itself. Hence the probability that the algorithm outputs X is

$$p(h) \geq \frac{1}{h+2} = \frac{1}{2\log_2 n + 2} = \Omega\left(\frac{1}{\log n}\right)$$

Improving the error bound

A single run of KARGER-STEIN succeeds with probability $\Omega(1/\log n)$ in time $O(n^2 \log n)$.

We repeat the algorithm independently $\Theta(\log n)$ times and return the smallest cut found. By the same argument as earlier (using $1 - x \leq e^{-x}$), this boosts the success probability to $\Omega(1)$ (constant probability). The overall running time of the algorithm is

$$O(\log n) \times O(n^2 \log n) = O(n^2 \log^2 n).$$

This is a substantial improvement over the $O(n^4)$ time of the basic algorithm.

Algorithm	Time per run	Total time for constant success probability
KARGER	$O(n^2)$	$O(n^4)$
KARGER-STEIN	$O(n^2 \log n)$	$O(n^2 \log^2 n)$

Table 1: Time complexities of the algorithms discussed

Problems and Solution Sketches

1. Prove that the worst-case time complexity of one run of Karger's min-cut algorithm is $O(n^2)$. Assume that we can pick an edge uniformly randomly in $O(\log m)$ time.

One run of Karger's min-cut algorithm involves $n-2$ iterations of the following steps:
1. Choosing an edge uniformly at random ($O(\log m)$, given)
2. Contracting the edge ($O(n)$, will show)

Thus, the worst case time complexity of one run of Karger's min-cut algorithm is $O(n) \times O(n + \log m) = O(n^2)$.

Edge Contraction: Using an adjacency matrix, it can be implemented in $O(n)$ (show).

2. Extend Karger's Min-Cut algorithm for weighted undirected graphs with positive edge weights without changing its worst-case time complexity.

Let $G = (V, E, w)$ be an undirected graph with positive edge weights $w : E \to \mathbb{R}_{>0}$, given as an $n \times n$ weighted adjacency matrix (so there are no parallel edges or self-loops). The **weight** of a cut U is $w(U) = \sum_{\{x,y\} \in E, x \in U, y \notin U} w(\{x, y\})$, and a **min-cut** is a cut of minimum weight.

Modify the Karger's min-cut algorithm as follows:

Instead of choosing the edge uniformly at random, assign probabilities for each edge to be chosen as follows:

$$p(e) = \frac{w_e}{\sum_{e \in E} w_e} = \frac{w_e}{W}$$

So, the modified algorithm is: iteratively choose an edge by sampling from this probabilty distribution, and the contract that edge. When only two vertices remain, return the corresponding cut.

- In this modified version, similarly choosing an edge and contracting the edge can be implemented in $O(n)$ time (show). Thus, one run of the algorithm runs in $O(n^2)$.

Error bound:
- The cuts of G/e correspond to the cuts of G that are not killed by e, with the same weights.
- If the min-cut has weight k, every vertex has *weighted degree* at least k, so $W \geq nk/2$ when n vertices remain.
- When i vertices remain and X has survived so far, the probability that the next contracted edge kills X is at most $2/i$.
- The probability that X is returned is at least $2/(n(n-1))$.

Thus, by the same analysis we can repeat the algorithm for $\Theta(n^2)$ times to get constant success probability. Therefore the algorithm runs in worst case $O(n^4)$.